

연금 Agent 멀티에이전트 아키텍처 검토

두 레포지토리 분석 및 우리 팀의 다음 단계

CONTENTS

1 두 레포지토리 분석

2 지금 우리는 어디에 있나

3 아키텍처 제안

4 다음 스텝 (우선순위)

5 정리

1. 두 레포지토리 분석

FinGuard AI (AI-Festival-2025-Fin-Agent)

제9회 2025 미래에셋증권 AI Festival **Tech 부문 최우수상** 수상작. "개인투자자의 정보 격차 해소를 위한 Multi-Agent 기반 투자 의사결정 지원 시스템".

레포 6개로 구성된다.

레포	역할	스택
Multi-Agent	오케스트레이터 (A2A)	FastAPI + ChatClovaX(HCX-007)
Flow-Agent	주가 검색·기술적 지표	LangGraph + SQLite × 4 + Text2SQL
Insight-Agent	공시 분석	LangGraph + OpenDART API
Rumor-Agent	뉴스·공시 교차검증	LangGraph + NAVER News API
FinGuardAI	웹	Spring Boot (Java)
.github	소개	—

오케스트레이션 방식

`multi_agent.py` 598줄이 전부다. 구조는 의외로 단순하다.

```
사용자 질문
↓
LLM 분류기 (프롬프트 1개) → {"type": "공시|루머체크|주가검색|기타"}
↓
QUESTION_TYPE_TO_AGENT 딕셔너리로 매핑
↓
httpx로 해당 에이전트 포트에 HTTP POST
```

각 에이전트는 **독립 프로세스**로 다른 포트에 뜬다 (8000/6200/2025/10000). LangGraph Platform의

`/runs/wait` 엔드포인트를 쓴다.

핵심: 이건 라우터지 플래너가 아니다. 질의 하나 → 에이전트 하나. 분해도 없고 계획도 없다. "기타" 는 그냥 공시 에이전트로 보낸다.

에이전트 간 협업(A2A)은 **각 에이전트 안에 하드코딩**돼 있다. Insight Agent의 `summarize_graph.py` 를 보면:

```
def check_and_call_finagent_node(state):
    # LLM에게 "이 질문에 추가 데이터가 필요한가?" YES/NO로 물어봄
    # YES면 http://localhost:8000/search 호출
```

즉 A2A는 오케스트레이터가 조율하는 게 아니라, 에이전트가 필요하면 직접 옆 에이전트를 부른다.

각 에이전트 내부 (여기가 배울 점)

Flow Agent — 가장 정교하다. ReAct 패턴에 검증 루프를 붙였다.

```
agent_node (LLM이 도구 선택)
  ↓ should_continue()
tool_node (16개 도구 중 실행)
  ↓ should_clarify()
  ├── param_missing + retry<2 → clarifier_node (되물어보기)
  ├── success → generation_node
  └── error → END
```

도구 16개를 3층으로 나눴다: 기본조회 4개 / 조건검색 4개 / 기술분석 7개. **그리고 안 되면 Text2SQL로 떨어뜨린다.** 이게 영리하다 — "전날 대비 거래량 100% 증가" 같은 집계·비교는 미리 만든 도구로 커버가 안 되니까 LLM이 SQL을 직접 쓰게 한다.

Insight Agent — 병렬 검색 후 병합.

```
query_parser → T→ document_search (정기공시) T→ merge_results → END
                ↳ revision_search (수정공시)   ↳
```

Rumor Agent — 병렬 수집 → 문서 선별 → 생성.

```
analyze_query → T→ search_news          T→ format_results
                  ↳ search_publications  ↳
                                      ↓
                                select_documents (LLM이 볼 문서 고름)
                                      ↓
                                generate_response
```

`select_documents`가 중요하다. 검색 결과를 전부 LLM에 넣지 않고, **LLM이 먼저 어떤 문서를 열어볼지 고른다.** 2단계 검색이다.

데이터 계층 — 결정적인 지점

세 에이전트 전부 **구조화된 API와 SQLite만 쓴다.**

- Flow: `stock_info.db`, `market_index.db`, `technical_indicators.db` (사전 수집)
- Insight: OpenDART API가 내려준 JSON 5,000여 개를 파일로 저장
- Rumor: NAVER News API 실시간 호출

벡터 DB도, 임베딩도, RAG도 없다. 전 레포를 검색해봤지만 `chroma` / `faiss` / `embedding` / `vectorstore` 사용처가 하나도 없다. Insight Agent README에 `rag_pipeline_v1.ipynb`가 언급되지만 레포에 파일이 없다 — 프로토타입만 만들고 접었다는 뜻이다.

회사명 매칭은 `difflib.SequenceMatcher`, 문서 조회는 회사명+연도+분기로 JSON 파일 로드. **PDF를 파싱하지 않는다**. 원문 문서를 다루는 대신 API가 정제해준 데이터를 받는다.

평가 체계 (이건 꼭 따라 해야 한다)

`Flow-Agent/test_folder/query/`에 정답이 붙은 평가셋 430문항이 있다.

파일	문항	유형
<code>simple_queries_2.json</code>	85	단순 조회
<code>conditional_queries_2.json</code>	50	조건 검색
<code>signal_queries_2.json</code>	50	기술적 지표
<code>open_queries.json</code>	245	개방형

형식이 명확하다.

```
{"input_data": {"message": "2024-12-19에 등락률이 -10% 이하인 종목을 모두 보여줘"},  
  "expected_output": "코아시스템켄온, 월비스, 오리엔트바이오, ...",  
  "evaluation_type": "PASS_OR_FAIL"}
```

"조건별 검색 정확도 94%"라는 수상작 홍보 문구는 여기서 나온 숫자다. **측정 가능한 지표가 있었기 때문에 개선할 수 있었고, 발표에서 주장할 수 있었다.**

mirae_ai_festa (제8회, 팀 슈퍼통개미)

전년도(제8회) 출품작. 세대가 다르다.

- 파일 구조: Jupyter 노트북 4개 + 파이썬 스크립트 5개 (총 505줄)
- 기능 3가지: 테마 탐지 / 충동매매 방지 챗봇 COMMA / 재무제표 요약 fin-bot
- 파이프라인: 네이버 뉴스 크롤링 → 클러스터링 → HCX-003 요약 → 조언 생성
- 인증 방식이 구버전 (`X-NCP-CLOVASTUDIO-API-KEY` + `/testapp/v1/chat-completions/HCX-003`)
- `tuning_data.ipynb` — 파인튜닝을 시도한 흔적이 있다

에이전트 프레임워크가 전혀 없다. LangGraph도, 도구 호출도, 라우팅도 없다. 스크립트를 순서대로 실행하고 프롬프트로 결과를 만든다.

교훈은 이것이다. 두 레포의 차이는 아이디어가 아니라 **엔지니어링 성숙도**다. 1년 사이에 "프롬프트 잘 쓰기"에서 "상태 그래프 + 도구 + 평가셋"으로 넘어갔다.

2. 지금 우리는 어디에 있나

성숙도를 6단계로 놓고 보면.

단계	내용	우리	FinGuard	슈퍼통개미
L0	데이터 수집			
L1	정제·구조화	 우위		
L2	검색 계층	 2%		
L3	단일 에이전트			
L4	멀티 에이전트			
L5	서비스·평가			

L1에서 우리가 앞서는 이유

FinGuard의 Insight Agent는 "공시 분석"을 하지만 **원문 PDF를 열지 않는다**. OpenDART API가 정제해서 내려주는 JSON을 받아 쓴다. 편하지만 한계가 명확하다 — API가 제공하지 않는 항목은 접근할 수 없다.

우리는 그 반대다. 투자설명서 PDF 100개, 161MB를 직접 파싱해서

- 펀드표준코드·펀드명·작성기준일을 **100개 전부** 추출 (4개 운용사 서식 모두 통과)
- 수수료·보수율 표 4,549개를 구조를 유지한 채 마크다운으로 보존
- 스캔 PDF는 OCR로 186페이지 복구
- 법정 공통문구 2,721건을 중복 제거하고 `doc_ids`로 추적

이건 **API로는 얻을 수 없는 데이터**다. 판매보수 0.72%가 어느 클래스의 것인지, `종류C-P2` (퇴직연금)와 `종류C-Pe` (개인연금)의 차이가 뭔지는 투자설명서 원문에만 있다.

L2~L5에서 뒤처지는 이유

간단하다. **아직 아무것도 없다**.

- 임베딩 300/14,745 (2%)
- 에이전트 0개
- API 서버 없음
- 평가셋 없음

FinGuard는 오케스트레이터를 598줄로 만들었다. 우리가 지금 없는 건 어려운 게 아니라 **아직 시작하지 않은** 것이다. 반대로 우리가 가진 L1은 그들이 시도했다가 (`rag_pipeline_v1.ipynb`) 접은 것이다.

요약: 우리는 깊고 좁다. 그들은 넓고 얇다.

3. 아키텍처 제안

3-1. FinGuard의 라우터를 그대로 베끼면 안 되는 이유

그들의 오케스트레이터는 **단일 라벨 분류기**다. 질문 하나에 에이전트 하나.

주식 도메인에서는 이게 통한다. "삼성전자 증가 얼마야?"는 명백히 주가 질문이다. 하지만 **연금 상담 질의는 본질적으로 다중 흡이다.**

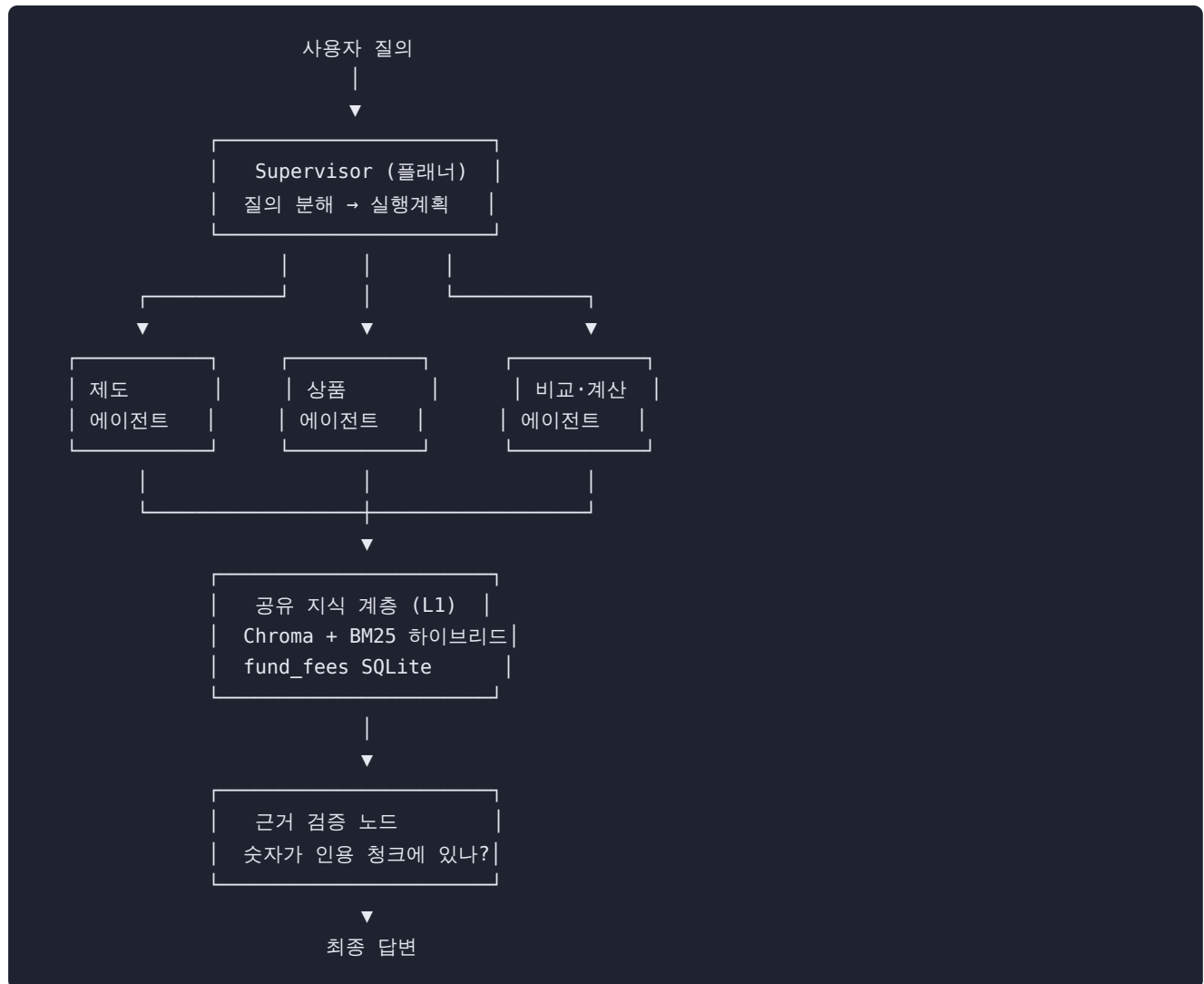
"IRP에 넣을 채권형 펀드 추천해줘. 수수료 싼 걸로."

이 한 문장이 요구하는 것:

1. IRP 가입 한도와 세액공제 규정 (제도 문서)
2. 채권형 + 퇴직연금 클래스 필터링 (투자설명서 메타데이터)
3. 총보수 기준 정렬·비교 (표 데이터 집계)
4. 위험등급 확인 (투자설명서)

단일 라벨 분류기로는 첫 단계에서 이미 틀린다. **우리에게 필요한 건 라우터가 아니라 플래너다.**

3-2. 제안 구조 — Supervisor + 공유 지식 계층



FinGuard와 다른 점 두 가지.

첫째, **지식 계층이 공유된다**. 그들은 에이전트마다 자기 DB를 갖고 HTTP로 통신한다. 우리는

`chunks_final.jsonl` 하나를 모든 에이전트가 도구로 호출한다. 에이전트는 "무엇을 어떻게 물을지"만 다르다.

둘째, **검증 노드가 있다**. 그들의 Rumor Agent는 "뉴스가 사실인가"를 검증한다. 연금 도메인에서 그에 대응하는 건 "내가 방금 만든 답변의 숫자가 실제 문서에 있는가"다. 수수료를 지어내면 그 순간 서비스가 끝난다. 이건 프롬프트로 부탁할 게 아니라 **답변의 모든 숫자를 인용 청크에서 문자열 매칭으로 검사하는 코드**여야 한다.

3-3. 각 에이전트

에이전트	담당	데이터	도구
제도	연금 제도·세제·한도·중도인출	<code>docs_renamed</code> 49건 + 공통 청크	하이브리드 검색
상품	개별 펀드 상세	투자설명서 100건	<code>--fund</code> 필터 검색
비교·계산	수수료 비교·정렬·집계	<code>fund_fees</code> SQLite	Text2SQL

3-4. 지금 당장 중요한 것 — 표를 SQL로 승격하라

이게 이번 분석에서 가장 실행 가치가 높은 결론이다.

지금 수수료 표는 **마크다운 텍스트 청크**로 저장돼 있다. 이러면 이런 질의를 못 푼다.

"퇴직연금 클래스 총보수가 0.5% 이하인 채권형 펀드 다 보여줘"

벡터 검색은 **비교·정렬·집계를 못 한다**. 의미가 비슷한 걸 찾을 뿐이다. "0.5% 이하"라는 조건을 만족하는 펀드 집합을 구하려면 SQL이 필요하다.

FinGuard가 Flow Agent에 Text2SQL을 넣은 이유가 정확히 이것이다. 도구 16개로도 "전날 대비 거래량 100% 증가" 같은 질의는 못 만들어서 LLM에게 SQL을 쓰게 했다.

우리 데이터에는 이미 재료가 있다. 표 청크 4,549개 중 상당수가 이 형태다.

구분	선취 판매수수료	후취 판매수수료	환매 수수료	운용 보수	판매 보수	신탁 보수	사무 관리
종류C-P2	수수료미징구-오프라인-퇴직연금	-	-	0.72	0.55	0.03	0.02

이걸 파싱해서 테이블로 만들면 된다.

```
CREATE TABLE fund_fees (
  fund_code TEXT, fund_name TEXT, base_date TEXT,
  class_code TEXT,          -- 'C-P2'
  class_desc TEXT,          -- '수수료미징구-오프라인-퇴직연금'
  channel TEXT,             -- '오프라인'
  pension_type TEXT,        -- '퇴직연금' | '개인연금' | NULL
  fee_front REAL, fee_back REAL, fee_redeem REAL,
  fee_mgmt REAL, fee_sales REAL, fee_trust REAL, fee_admin REAL,
  fee_total REAL,          -- 합산
  risk_grade INTEGER,
  PRIMARY KEY (fund_code, class_code)
);
```

RAG와 SQL은 경쟁 관계가 아니라 역할 분담이다.

- "중도인출하면 세금 얼마나 떼?" → 벡터/BM25 검색 (서술형 지식)
- "총보수 0.5% 이하 퇴직연금 클래스" → SQL (정형 조회)
- "이 펀드 왜 위험등급 1등급이야?" → 검색 + SQL 조합

3-5. A2A는 HTTP가 아니라 in-process로 시작하라

FinGuard는 포트 5개에 프로세스 5개를 띄운다. 규모가 커지면 맞는 선택이지만 지금 우리 팀에는 과하다. 디버깅이 5배로 어려워지고, 하나가 죽으면 `unreachable` 을 반환하며 조용히 실패한다 (그들의 `/health` 코드가 그 증거다).

LangGraph 단일 프로세스 + 서브그래프로 시작하자. 상태를 공유하고, 스택 트레이스가 한 곳에 찍히고, 배포가 하나다. 나중에 특정 에이전트가 무거워지면 그때 떼어내면 된다.

4. 다음 스텝 (우선순위)

지금 → 2주

- ① **임베딩 완료** — CLOVA Studio 서비스 앱 신청으로 호출 한도부터 올릴 것. 테스트 키로는 12시간이 걸린다.
- ② **평가셋 30~50문항** — FinGuard의 `test_folder/query/` 형식을 그대로 베끼자.

```
{
  "input_data": {
    "message": "퇴직연금 중도인출 사유가 뭐야?",
    "expected_chunks": ["doc20_p1_0004"],
    "expected_output": "무주택자 주택구입, 전세보증금, 6개월 이상 요양, ...",
    "evaluation_type": "PASS_OR_FAIL"
  }
}
```

이게 없으면 청크 크기를 900자에서 600자로 바꿨을 때 **좋아졌는지 나빠졌는지 알 수 없다.** 지금 우리에겐 "표 청크 4,549개"라는 숫자는 있어도 "정확도 몇 %"라는 숫자가 없다. 발표에서 주장할 수 있는 건 후자다.

- ③ **fund_fees 테이블 구축** — 표 청크 → SQLite. 위 3-4 참조.

3주 → 6주

- ④ **단일 에이전트 (L3)** — LangGraph StateGraph 하나. 도구는 `search_documents` (하이브리드 검색)와 `query_fees` (SQL) 둘로 시작. Flow Agent의 `agent → tool → should_clarify → generation` 패턴을 그대로 쓰면 된다.
- ⑤ **근거 검증 노드** — 답변 속 모든 숫자·비율이 인용 청크에 존재하는지 코드로 검사. 없으면 재생성하거나 "확인 불가"로 표시. 연금 도메인에서는 이게 차별점이 된다.

6주 →

⑥ **Supervisor + 도메인 분리 (L4)** — 단일 에이전트가 도구 10개를 넘어가고 프롬프트가 뒤엉키기 시작할 때 쪼개면 된다. **먼저 쪼개지 말 것.**

⑦ **API 서버 + 웹 (L5)**

5. 정리

우리 강점		우리 약점
데이터	PDF 원문 파싱, 표 보존, 메타데이터 완비	—
검색	하이브리드 설계 완료	아직 2%만 인덱싱
에이전트	—	없음
평가	—	없음

FinGuard에서 가져올 것 세 가지.

1. **평가셋 문화** — 430문항에 정답을 붙였기 때문에 "94%"를 말할 수 있었다
2. **Text2SQL 풀백** — 미리 만든 도구로 안 되는 질의는 SQL로 떨어뜨린다
3. **LangGraph 노드 분리** — 검색·선별·생성을 노드로 나누면 어디서 틀렸는지 보인다

가져오지 말 것 두 가지.

1. **단일 라벨 라우터** — 연금 질의는 다중 홉이라 플래너가 필요하다
2. **초기부터 HTTP 마이크로서비스** — 팀 규모에 비해 운영 비용이 크다

우리가 그들보다 나은 지점은 명확하다. **원문 문서를 다룬다는 것.** API가 주는 데이터로는 "종류C-P2의 판매보수가 0.55%"라고 답할 수 없다. 그 강점을 살리려면 검색 계층을 끝내고, 표를 SQL로 올리고, 추정을 시작해야 한다. 에이전트는 그다음이다.